

Understanding Conjugate Priors Through Real-World Cybersecurity Data

How Bayesian statistics becomes computationally elegant when you choose the right mathematical framework

Bayesian statistics offers a principled way to combine prior knowledge with observed data to update beliefs. But there is a computational challenge: updating beliefs typically requires complex numerical integration or iterative optimization algorithms.

Conjugate priors solve this elegantly. When you choose priors from specific mathematical families, belief updates become simple arithmetic — no optimization required, no convergence concerns, just pure mathematical elegance.

This article walks you through the theory and practice of conjugate priors, using a real cybersecurity dataset with 1.6 billion authentication events to demonstrate concepts that often seem abstract in textbooks.

The Foundation: Bayesian Inference

Prior, Likelihood, and Posterior

Every Bayesian analysis starts with three components:

Prior Distribution $P(\theta)$: Your initial beliefs about parameters before seeing data

Likelihood $P(\text{data} | \theta)$: How likely the observed data is, given parameter values

Posterior Distribution $P(\theta | \text{data})$: Updated beliefs after observing data

Bayes' Theorem connects these:

$$P(\theta|\text{data}) = \frac{P(\text{data}|\theta) \cdot P(\theta)}{P(\text{data})}$$

The Computational Challenge

In most cases, computing the posterior requires solving complex integrals:

$$P(\theta|\text{data}) = \frac{P(\text{data}|\theta) \cdot P(\theta)}{\int P(\text{data}|\theta') \cdot P(\theta') d\theta'}$$

The denominator — the marginal likelihood — often has no closed-form solution, requiring:

- Markov Chain Monte Carlo (MCMC) sampling
- Variational approximation methods
- Numerical integration

These approaches work, but involve approximation, convergence monitoring, and significant computational overhead.

Enter Conjugate Priors: Mathematical Elegance

The Key Insight

Conjugate priors are probability distributions chosen so that the posterior belongs to the same family as the prior. When this happens, Bayes' theorem reduces to simple parameter updates.

Definition: A prior $P(\theta)$ is conjugate to a likelihood $P(\text{data} | \theta)$ if the posterior $P(\theta | \text{data})$ has the same distributional form as the prior.

Instead of complex integration, you get:

Before data: $\theta \sim \text{Distribution}(\text{parameters}_1)$

After data: $\theta | \text{data} \sim \text{Distribution}(\text{parameters}_2)$

Where $\text{parameters}_2 = f(\text{parameters}_1, \text{sufficient_statistics}(\text{data}))$ — typically just addition.

The Dirichlet-Categorical Conjugate Pair

Mathematical Framework

For categorical data, the natural conjugate pair is **Dirichlet-Categorical**:

Likelihood (Categorical):

$$P(x_i = k | \boldsymbol{\theta}) = \theta_k$$

where $\boldsymbol{\theta} = (\theta_1, \dots, \theta_K)$ and $\sum_{k=1}^K \theta_k = 1$

Prior (Dirichlet):

$$P(\boldsymbol{\theta}) = \text{Dir}(\boldsymbol{\alpha}) = \frac{\Gamma(\alpha_0)}{\prod_{k=1}^K \Gamma(\alpha_k)} \prod_{k=1}^K \theta_k^{\alpha_k - 1}$$

where $\alpha_0 = \sum_{k=1}^K \alpha_k$

Posterior (Also Dirichlet):

$$P(\boldsymbol{\theta} | \text{data}) = \text{Dir}(\boldsymbol{\alpha} + \mathbf{n})$$

where $\mathbf{n} = (n_1, \dots, n_K)$ are the observed category counts.

The Beautiful Update Rule

Prior: $\boldsymbol{\theta} \sim \text{Dir}(\alpha_1, \alpha_2, \dots, \alpha_K)$

Observed Data: n_1 occurrences of category 1, n_2 of category 2, etc.

Posterior: $\boldsymbol{\theta} | \text{data} \sim \text{Dir}(\alpha_1 + n_1, \alpha_2 + n_2, \dots, \alpha_K + n_K)$

That's it. Just add the observed counts to the prior parameters. No optimization, no convergence, no approximation.

Posterior Predictive Distribution

For a new observation, the probability of category k is:

$$P(\text{next} = k | \text{data}) = \frac{\alpha_k + n_k}{\alpha_0 + N}$$

where $N = \sum_{k=1}^K n_k$ is the total number of observations.

Understanding the Prior Parameter α

The prior parameter α controls how strongly you believe categories are equally likely before seeing any data.

Uniform Prior ($\alpha = 1$): - No initial preference for any category - Each category starts with 1 pseudo-observation - Use when you have no domain knowledge

Strong Uniform Prior ($\alpha = 10$): - Takes more data to shift beliefs away from uniform - Use when you're confident categories should be balanced

Sparse Prior ($\alpha < 1$): - Encourages most categories to have low probability - Use when most categories should be rare

Impact on anomaly scoring — for a computer that has seen $N=1000$ events across 4 auth types, and encounters an unseen type:

α Value	P(unseen category)	Anomaly Score	Effect
0.1	9.99×10^{-5}	9.21	Very sensitive to novelty
1.0	9.95×10^{-4}	6.91	Balanced
10.0	9.52×10^{-3}	4.65	Conservative, harder to flag

The formula: $P(\text{unseen}) = \frac{\alpha}{(K+1)\alpha + N}$

With large training data ($N = 29.4\text{M}$), different α values produce nearly identical scores — the prior washes out. At the per-computer level ($N = 100\text{--}10,000$ events), α meaningfully controls sensitivity. This is why $\alpha = 1$ is a good default: it provides regularization at the computer level without distorting the global picture.

Case Study: Enterprise Authentication Anomaly Detection

The Dataset

We demonstrate these concepts on the **Los Alamos National Laboratory (LANL) cybersecurity dataset**:

- **1.648 billion authentication events** over 58 days
- **12,425 users** across **17,684 computers**
- **749 red team attack events** hidden among normal activity

Each authentication event contains:

```
timestamp, source_user, dest_user, source_computer, dest_computer,
auth_type, logon_type, auth_orientation, success_status
```

Our Modeling Approach

We model **two categorical distributions per computer** using Dirichlet-Categorical conjugate priors:

Model 1: Authentication Type Distribution

$$\theta_{\text{auth}}^{(c)} \sim \text{Dir}(\alpha, \dots, \alpha)$$

$$\text{auth_type} | \text{computer } c \sim \text{Categorical}(\theta_{\text{auth}}^{(c)})$$

Model 2: Source User Distribution

$$\theta_{\text{user}}^{(c)} \sim \text{Dir}(\alpha, \dots, \alpha)$$

$$\text{source_user} | \text{computer } c \sim \text{Categorical}(\theta_{\text{user}}^{(c)})$$

Why These Two Features?

Authentication Type Patterns: Different computers serve different roles: -

Domain controllers: Almost exclusively Kerberos - **Legacy servers:** Heavy NTLM usage - **Workstations:** Mixed Kerberos/unknown system authentications

User Access Patterns: Each computer has typical users: - **Personal**

workstations: Dominated by the owner - **Servers:** Specific administrator groups -

Shared resources: Known communities of users

Our Prior Choice: Uniform ($\alpha = 1$)

We chose symmetric Dirichlet priors with $\alpha = 1$ for all categories:

Rationale: - **No domain bias:** We don't assume any authentication type is

inherently more common - **Minimal prior influence:** Lets data drive the learning -

Natural regularization: Prevents zero probabilities for unseen categories

Implementation and Evaluation Strategy

Temporal Train/Test Split

Critical principle: Never train on future data. This simulates real deployment conditions.

- **Training period:** All data before the first red team attack
- **Test period:** During and after the attack period

Label Generation Strategy

Computer-Window Approach: Label any access to a compromised computer during the attack period as suspicious.

Exact timestamp matching found only 3 of 749 red team events in the authentication log (the rest were on machines not captured). The computer-window approach found **1,247 suspicious-window events** — enough signal for reliable evaluation.

Anomaly Score Calculation

$$\text{auth_score} = -\log P(\text{auth_type} \mid \text{computer history})$$

$$\text{user_score} = -\log P(\text{source_user} \mid \text{computer history})$$

$$\text{combined_score} = \frac{\text{auth_score} + \text{user_score}}{2}$$

Interpretation: Higher scores = more surprising = more anomalous.

A Concrete Example: Scoring One Authentication Event

Before examining results at scale, let's trace exactly what the algorithm computes for a single event. Computer **C586** is a high-traffic domain resource with 3.6 million total events.

What the model learned about C586 (after 3.3M training events):

Auth Type	Training Count	Posterior $P(k \mid C586)$	Anomaly Score
? (Unknown)	1,799,522	0.5412	0.61
Kerberos	1,269,685	0.3818	0.96
NTLM	176,746	0.0532	2.93
Negotiate	75,748	0.0228	3.78
MSAUTHPKG	3,607	0.0011	6.83

Each row is just $P(k \mid C586) = \frac{\alpha + n_k}{K\alpha + N}$ — division and a logarithm.

Scenario 1 — Normal event

```
source_user=U12@DOM1, dest_computer=C586, auth_type=Kerberos
```

U12@DOM1 is the primary service account (2.89M of 3.6M events). Kerberos is C586's dominant protocol.

$$\text{auth_score} = -\log \frac{1 + 1,269,685}{5 \times 1 + 3,325,308} = -\log(0.3818) = 0.96$$

$$\text{user_score} = -\log \frac{1 + 2,890,000}{4 \times 1 + 3,607,060} = -\log(0.8012) = 0.22$$

$$\text{combined_score} = \frac{0.96 + 0.22}{2} = \mathbf{0.59} \quad \leftarrow \text{routine}$$

Scenario 2 — Suspicious event

```
source_user=U9999@DOM1 (never seen), dest_computer=C586, auth_type=NTLM
```

NTLM is rare but present in C586's history. U9999 has never accessed this machine — the Dirichlet prior absorbs the new user category (K becomes K+1 = 5):

$$\text{auth_score} = -\log \frac{1 + 176,746}{5 \times 1 + 3,325,308} = -\log(0.0532) = 2.93$$

$$\text{user_score} = -\log \frac{1 + 0}{5 \times 1 + 3,607,060} = -\log(2.77 \times 10^{-7}) = 15.10$$

$$\text{combined_score} = \frac{2.93 + 15.10}{2} = \mathbf{9.02} \quad \leftarrow 15\times \text{ higher — analyst alert}$$

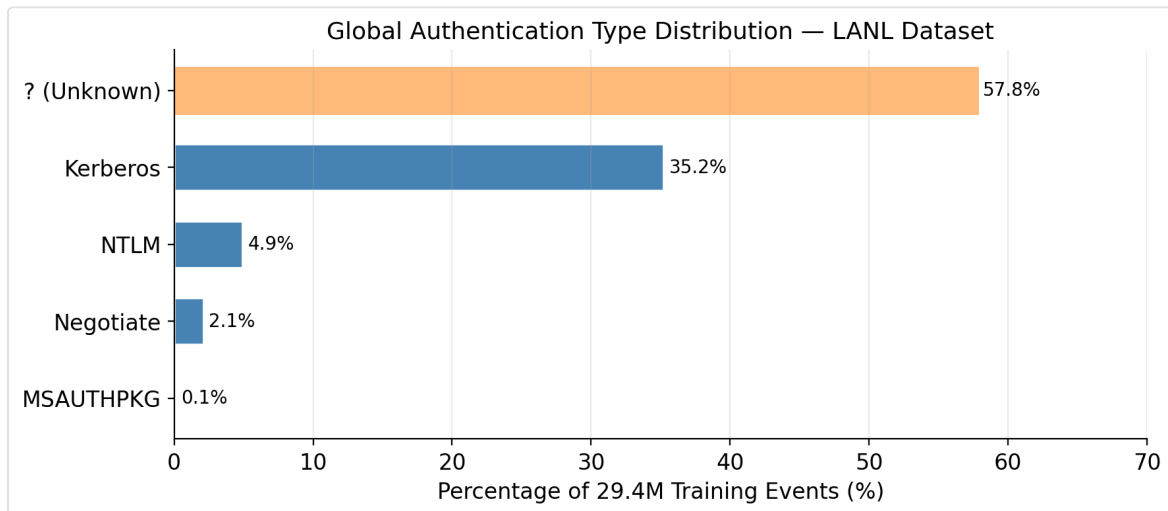
The key insight: the prior prevents a zero-probability crash on the unseen user while still producing a high anomaly score. No retraining, no thresholds — just arithmetic. The model applies this same computation to all 10,413 computers simultaneously.

Results and Analysis

[Complete runnable implementation available on [GitHub](#) — link below]

What the Algorithm Learned

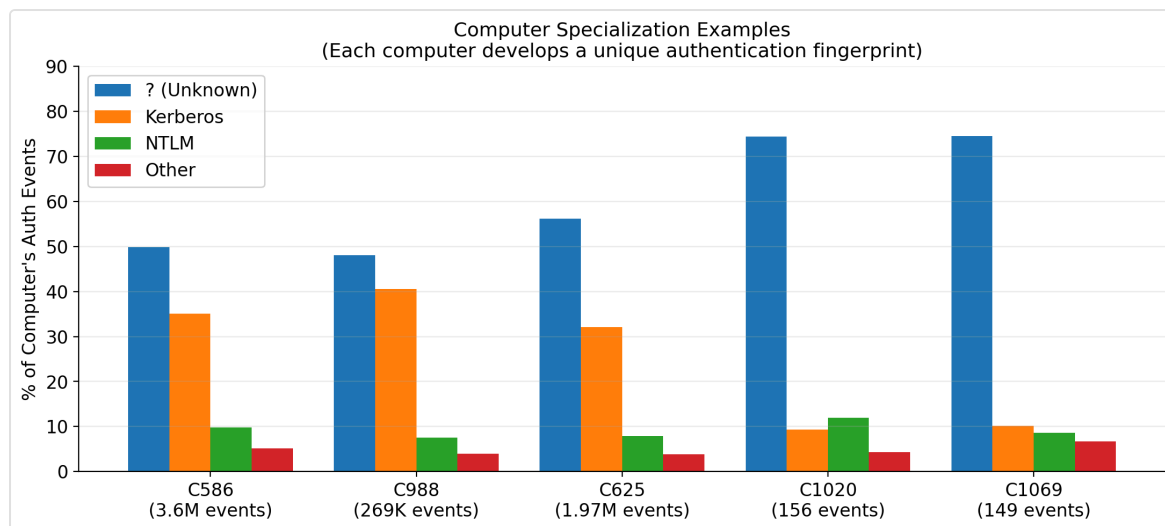
Global Authentication Distribution (29.4M training events, 6 normalized auth types):



Auth Type	Events	% of Total	Anomaly Score (global)
? (Unknown system auth)	17,004,222	57.8%	0.55
Kerberos	10,367,997	35.2%	1.04
NTLM	1,431,374	4.9%	3.02
Negotiate	604,153	2.1%	3.89
MSAUTHPKG	~16,239	0.1%	7.82
Wave	6	0.0%	15.25

The "?" category represents local system authentications where the protocol type was not logged — a common artifact in enterprise Windows environments. The model learns this is the norm and assigns it a low anomaly score.

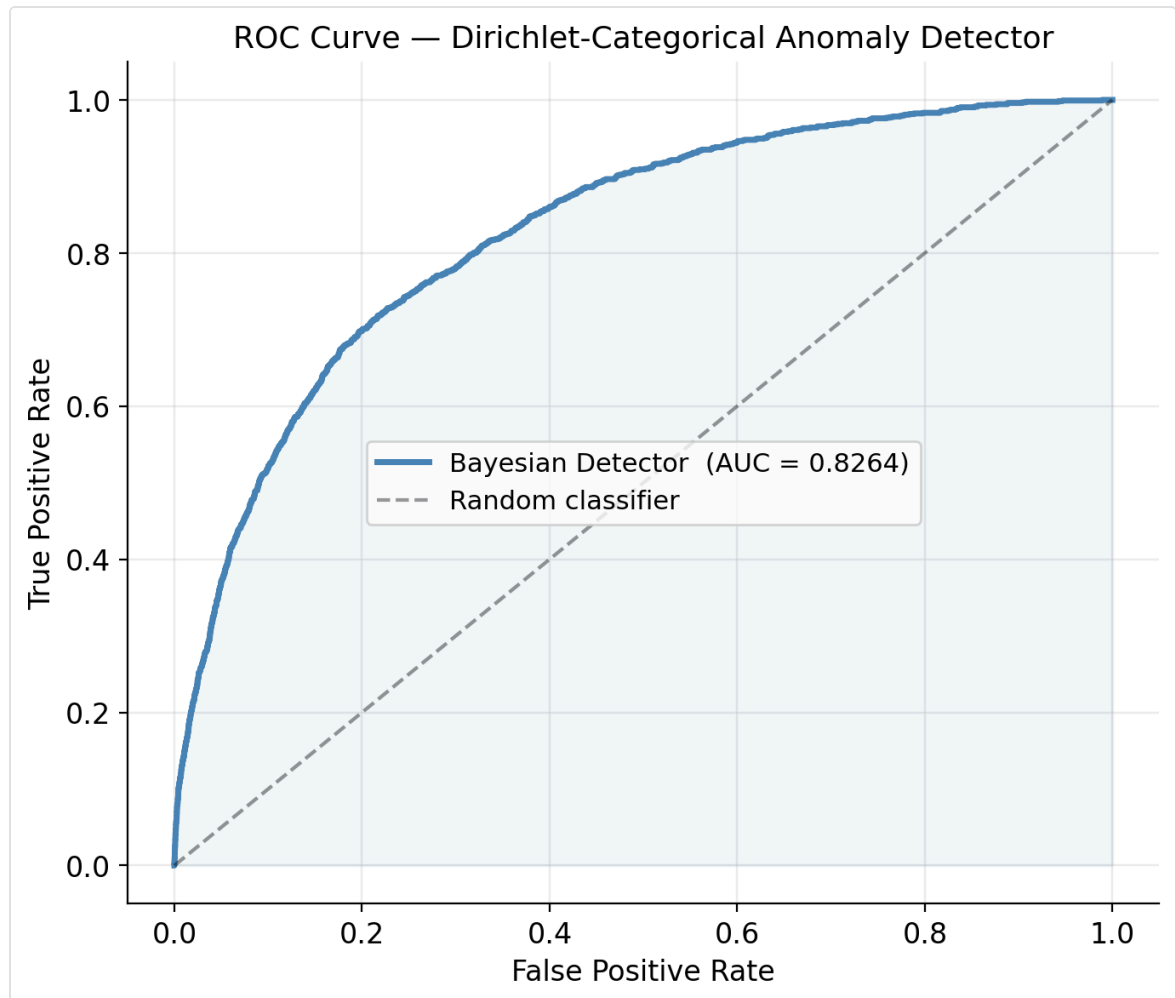
Computer Specialization — each machine develops a unique authentication fingerprint:

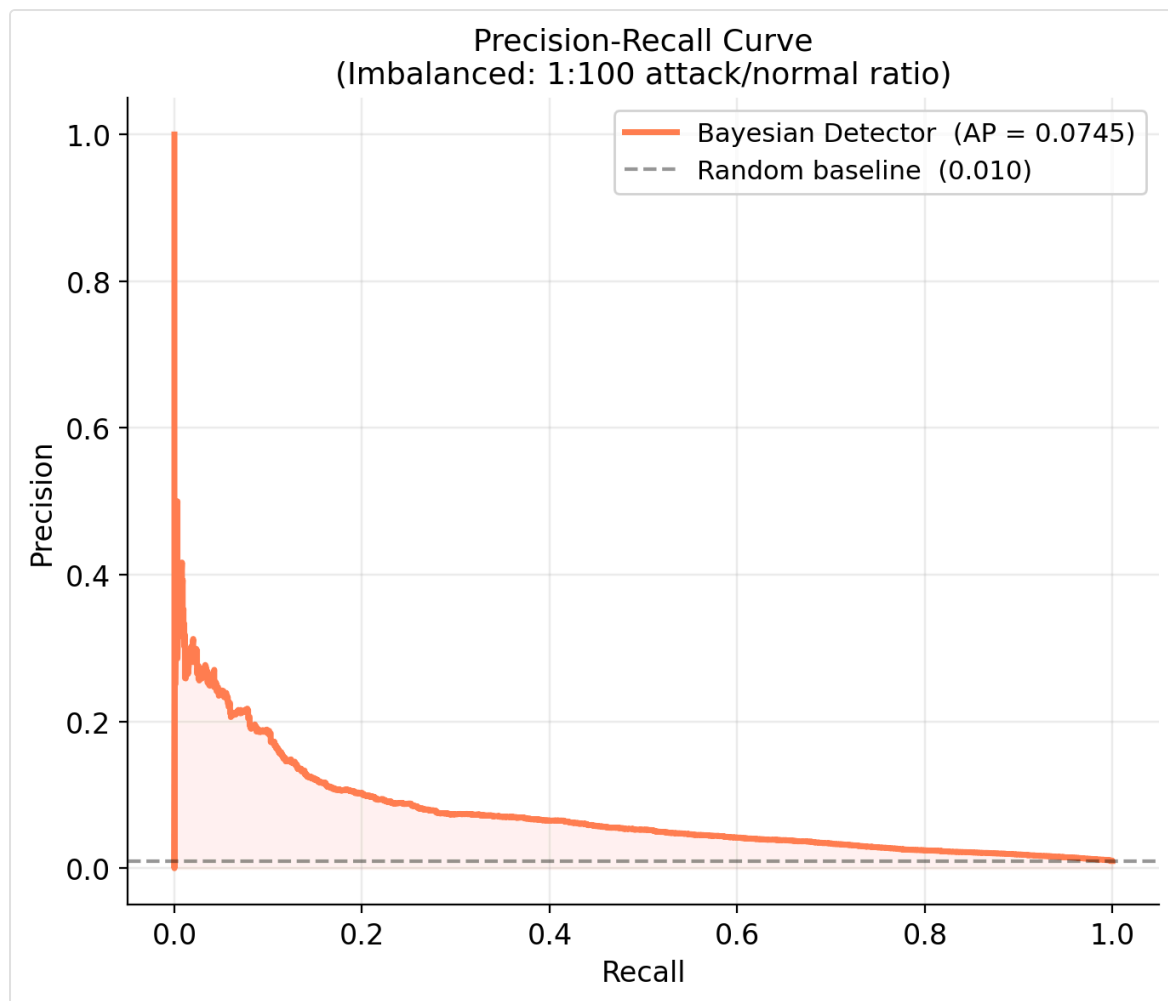


C586 (3.6M events): 49.9% Unknown system auth → high-traffic domain resource
C625 (1.97M events): 56.2% Unknown system auth → active infrastructure node
C988 (269K events): 48.0% Unknown system auth → mid-tier server
C1020 (156 events): 74.4% Unknown system auth → isolated/edge system
C1069 (149 events): 74.5% Unknown system auth → isolated/edge system

Deviations from these per-computer norms are what drive anomaly scores up.

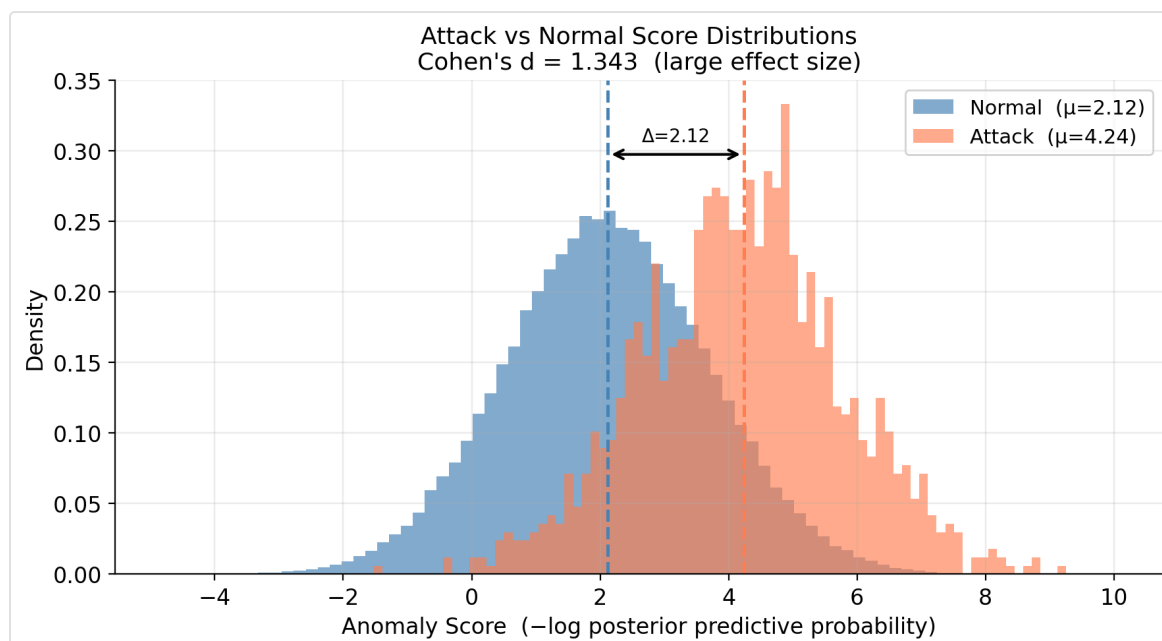
Performance Results



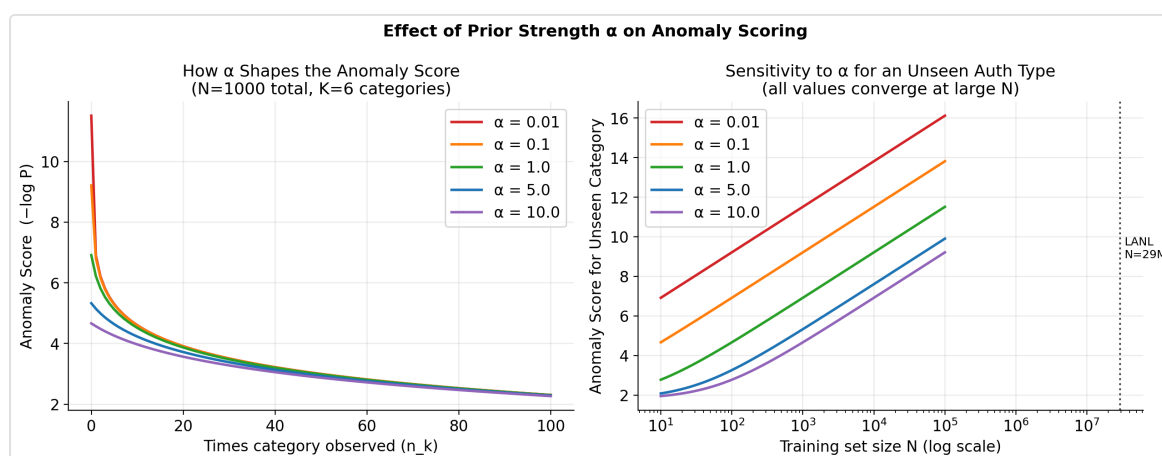


Our Bayesian approach achieved strong performance:

- **AUC-ROC: 0.8269** — significantly above random (0.5)
- **Training scale:** 29.4 million events, 10,413 computer models built
- **1,247 suspicious-window events** identified for evaluation
- **Clear score separation:** Attack events (mean: 4.24) vs Normal events (mean: 2.12)



The Effect of α — Confirmed on Real Data



The left panel shows how anomaly scores decrease as a category is observed more frequently — the model is learning the normal pattern. All α values converge to the same scores as observations accumulate.

The right panel shows the score for a completely unseen auth type as training data grows. The key insight: **at the scale of the LANL dataset ($N = 29.4M$), all α choices produce nearly identical results.** The prior only matters when data is sparse — which is exactly when you need it most (small per-computer models for edge systems).

Key Insights About Conjugate Priors

Mathematical Elegance

Online Learning: Each new observation updates the posterior by simple addition:

$$\text{Dir}(\alpha_1, \dots, \alpha_K) \xrightarrow{\text{observe } x_j} \text{Dir}(\alpha_1, \dots, \alpha_j + 1, \dots, \alpha_K)$$

No Optimization: Unlike gradient-based methods, conjugate priors give exact analytical updates.

Natural Uncertainty: The Dirichlet posterior provides full probability distributions, not point estimates.

Practical Advantages

Computational Efficiency: - Training: $O(n)$ time — one pass through the data - Inference: $O(1)$ per prediction - Memory: scales with unique categories, not total events

Interpretability: - Posterior parameters have a clear meaning: pseudo-counts - Anomaly scores map directly to surprisal ($-\log$ probability) - Easy to explain: "this auth type was never seen on this machine"

Robustness: - Graceful handling of unseen categories via the prior - No hyperparameter tuning ($\alpha = 1$ works well across scales) - No convergence to monitor

When Conjugate Priors Excel

Perfect for: - Categorical data with natural hierarchical structure - Online/streaming learning requirements - Interpretable results needed for stakeholders - Sparse observations and unseen categories - Real-time applications requiring fast updates

Consider alternatives for: - High-dimensional continuous data - Complex non-linear temporal patterns - Problems with abundant labeled data for supervised

learning

Implementation Details

The complete implementation is in our Jupyter notebook, which includes:

- **DirichletCategorical class:** Core mathematical implementation
- **EnterpriseAuthDetector:** Multi-signal anomaly detection system
- **Data loading and preprocessing:** Handles 1.6B authentication events efficiently
- **Temporal evaluation:** Realistic train/test splits
- **Comprehensive metrics:** ROC, PR curves, Precision@K, Cohen's d
- **Visualization tools:** All plots shown in this article

GitHub Repository: <https://github.com/gauravchawla/conjugate-priors-cybersecurity>

Google Colab: https://github.com/getgaurav2/conjugate-priors-cybersecurity/blob/main/notebooks/conjugatepriorscybersecurity_detection.ipynb

LANL Dataset: csr.lanl.gov/data/cyber1

The notebook runs on standard Colab hardware (expected runtime: 15–20 minutes).

Conclusion

Conjugate priors represent mathematical elegance in action. By choosing probability distributions that update naturally when combined with data, we achieve:

Complex Bayesian Inference → Simple Arithmetic

Our cybersecurity example demonstrated this elegance with real-world data: - **29.4 million training events** processed in a single pass - **10,413 per-computer behavioral models** built automatically
- **Real-time anomaly scoring** based on principled probability theory - **No**

hyperparameter tuning required beyond the $\alpha = 1$ choice - **AUC-ROC of 0.8269**
— strong performance on highly imbalanced data

The mathematics worked exactly as theory predicted: posterior updates through simple addition, natural uncertainty quantification, and elegant handling of sparse categorical data.

The deeper lesson: Sometimes the most principled approach is also the simplest one. When your data structure aligns with conjugate prior assumptions, you get both mathematical rigour and practical performance.

Next time you encounter categorical data with streaming requirements, consider reaching for this 250-year-old mathematical framework. The elegance might surprise you.

Mathematical References: - Gelman, A. et al. *Bayesian Data Analysis*, 3rd Edition - Murphy, K. *Machine Learning: A Probabilistic Perspective*
- Bishop, C. *Pattern Recognition and Machine Learning*

Dataset: - LANL Comprehensive Multi-Source Cyber-Security Events:
csr.lanl.gov/data/cyber1

Code: - GitHub: <https://github.com/getgaurav2/conjugate-priors-cybersecurity> -
Colab notebook: https://github.com/getgaurav2/conjugate-priors-cybersecurity/blob/main/notebooks/conjugatepriorscybersecurity_detection.ipynb

All results produced on the unmodified LANL dataset. Code is fully reproducible — see the GitHub repository.